JZ-HDL WAVEFORM FORMAT SPECIFICATION (JZW)

State: Beta — Version: 0.1.8

Contents

1. OVERVIEW

This specification defines the **JZW (JZ-HDL Waveform)** file format, a SQLite-based waveform storage format for JZ-HDL simulation output. JZW is the native waveform format for the JZ-HDL toolchain.

Motivation: Standard waveform formats (VCD, FST) store only signal value changes. They have no mechanism for timestamped annotations, colored markers, conditional alerts, or any form of simulation metadata beyond the raw signal data. JZW extends value-change storage with first-class support for annotations, enabling richer simulation analysis in JZ-HDL viewers.

Design Principles:

- **SQLite as the container.** JZW files are standard SQLite 3 databases. Any SQLite client can open, query, and inspect them. No custom binary parser is required.
- **Change-based storage.** Only value changes are recorded. If a signal does not change at a given time, no row is written. Time 0 provides an initial value for every signal, establishing the baseline.
- **Picosecond resolution.** All time values are stored as 64-bit unsigned integers representing picoseconds, consistent with the simulation engine's internal representation (Simulation Specification Section 2.1). There is no timescale ambiguity.
- **Annotation-native.** Alerts, markers, and selection ranges are stored alongside value changes in dedicated tables, tied to the same picosecond timeline.
- **Streaming-compatible.** Using SQLite WAL (Write-Ahead Logging) mode, a viewer can read the database while the simulator is still writing to it, enabling real-time waveform display.

Relationship to VCD and FST:

JZW does not replace VCD or FST output. The simulator supports all three formats via CLI flags (`--vcd`, `--fst`, `--jzw`), with one output format selected per simulation run. VCD and FST remain available for interoperability with third-party tools (e.g., GTKWave).

2. FILE STRUCTURE

2.1 File Extension

JZW files use the `.jzw` extension.

2.2 SQLite Version

JZW files must be valid SQLite 3 databases. The minimum required SQLite version is 3.8.0 (for common table expression support and WAL mode reliability). The database must use UTF-8 encoding.

2.3 Journal Mode

The simulator opens the database in **WAL (Write-Ahead Logging)** mode. This allows concurrent read access from a viewer process while the simulation is running. The simulator sets `PRAGMA journal_mode=WAL` immediately after creating the database.

2.4 Creation Sequence

When the simulator creates a JZW file:

1. Create the SQLite database file.
2. Set `PRAGMA journal_mode=WAL`.
3. Set `PRAGMA synchronous=NORMAL` (balance between durability and write performance).
4. Create all tables and indexes (Section 3).
5. Populate the `meta` table (Section 3.1).
6. Populate the `signals` table with all monitored signals (Section 3.2).
7. Populate the `clocks` table with clock configuration and runtime parameters (Section 3.5).
8. Begin simulation, writing value changes and annotations as the simulation progresses.
9. On simulation completion, write the `sim_end_time` metadata key and close the database.

3. DATABASE SCHEMA

3.1 Metadata Table

```
CREATE TABLE meta (
    key   TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
```

The `meta` table stores key-value pairs describing the simulation context. The following keys are defined:

Key	Required	Description
`jzw_version`	Yes	Schema version. This specification defines version `1`.
`timescale`	Yes	Always `1ps`. Informational; confirms that all time values are in picoseconds.
`sim_start_time`	Yes	Simulation start time in picoseconds. Always `0`.
`sim_end_time`	Yes	Simulation end time in picoseconds. Written when the simulation completes. If the simulation is still running or was aborted, this key may be absent.
`date`	Yes	ISO 8601 date-time when the simulation was run (e.g., `2026-03-09T14:30:00Z`).
`source_file`	Yes	Path to the JZ-HDL source file that was simulated.
`compiler_version`	Yes	JZ-HDL compiler version string (e.g., `0.1.0`).
`seed`	Yes	Simulation seed in hexadecimal (e.g., `0xDEADBEEF`).
`tick_ps`	Yes	Tick resolution in picoseconds (the GCD of all clock half-periods).
`module_name`	Yes	Name of the top-level module under simulation.

Implementations may add additional keys. Readers must ignore unrecognized keys.

3.2 Signals Table

```
CREATE TABLE signals (
    id    INTEGER PRIMARY KEY,
    name  TEXT    NOT NULL,
    scope TEXT    NOT NULL,
    width INTEGER NOT NULL,
    type  TEXT    NOT NULL
);
```

Each row defines a monitored signal. Signal IDs are assigned sequentially starting from 1.

Column	Description
`id`	Unique signal identifier. Referenced by the `changes` and `annotations` tables.
`name`	Signal name without hierarchy (e.g., `wr_ptr`).
`scope`	Hierarchical scope path using dot separators (e.g., `clocks`, `wires`, `dut`, `dut.sub`). Matches the VCD scope grouping defined in Simulation Specification Section 6.2.
`width`	Bit width of the signal (e.g., 1, 8, 32).
`type`	Signal type. One of: `clock`, `wire`, `tap`.

Scope conventions:

- Signals from the `CLOCK` block have scope `clocks`.
- Signals from the `WIRE` block have scope `wires`.
- Signals from the `TAP` block use the instance hierarchy as scope (e.g., `TAP { dut.wr_ptr; }` produces scope `dut`, name `wr_ptr`).

3.3 Value Changes Table

```
CREATE TABLE changes (
    time      INTEGER NOT NULL,
    signal_id INTEGER NOT NULL,
    value     TEXT    NOT NULL,
    FOREIGN KEY (signal_id) REFERENCES signals(id)
);

CREATE INDEX idx_changes_time ON changes(time);
CREATE INDEX idx_changes_signal ON changes(signal_id, time);
```

Each row records a single value change for a single signal at a specific time.

Column	Description
`time`	Simulation time in picoseconds when the change occurred.

Column	Description
`signal_id`	References `signals.id`.
`value`	The new signal value as a text string (see Section 4).

Rules:

- **Time 0 baseline.** At time 0, exactly one row must be written for every signal in the `signals` table. This establishes the initial value of every signal before any clock edge fires.
- **Change-only storage.** After time 0, a row is written for a signal only when its value differs from its most recent recorded value. If a signal holds the same value across consecutive ticks, no row is written.
- **Monotonic time.** Rows are inserted in non-decreasing time order. Multiple signals may change at the same time, but a given signal may have at most one change per time value.

3.4 Annotations Table

```sql
CREATE TABLE annotations (
    id        INTEGER PRIMARY KEY,
    time      INTEGER NOT NULL,
    type      TEXT    NOT NULL,
    signal_id INTEGER,
    message   TEXT,
    color     TEXT,
    end_time  INTEGER,
    FOREIGN KEY (signal_id) REFERENCES signals(id)
);

CREATE INDEX idx_annotations_time ON annotations(time);
```

Annotations are timestamped metadata entries associated with the simulation timeline. They are generated by annotation directives in the simulation source (Section 5).

Column	Description
`id`	Auto-incrementing unique identifier.
`time`	Simulation time in picoseconds when the annotation was created.
`type`	Annotation type string. One of: `alert`, `mark`, `select`, `trace`.
`signal_id`	Optional. References `signals.id`. If non-NULL, the annotation is associated with a specific signal. If NULL, it is a global annotation (applies to the entire time column).
`message`	Optional. Human-readable text describing the annotation.
`color`	Optional. Display color hint. One of the predefined color names (Section 5.4).

Column	Description
`end_time`	Optional. For range annotations (e.g., `select`), the end time in picoseconds. NULL for point-in-time annotations.

3.5 Clocks Table

```
CREATE TABLE clocks (
    id                INTEGER PRIMARY KEY,
    name              TEXT    NOT NULL,
    period_ps         INTEGER NOT NULL,
    phase_ps          INTEGER NOT NULL DEFAULT 0,
    jitter_pp_ps      INTEGER NOT NULL DEFAULT 0,
    jitter_sigma_ps   REAL    NOT NULL DEFAULT 0.0,
    drift_max_ppm     REAL    NOT NULL DEFAULT 0.0,
    drift_actual_ppm  REAL    NOT NULL DEFAULT 0.0,
    drifted_period_ps REAL    NOT NULL DEFAULT 0.0
);
```

The `clocks` table stores the configuration and runtime parameters of each clock in the simulation. One row is written per clock at simulation initialization, before any value changes are recorded.

Column	Description
`id`	Auto-incrementing unique identifier.
`name`	Clock name as declared in the `CLOCK` block (e.g., `clk_wr`).
`period_ps`	Nominal full period in picoseconds (twice the half-period / toggle interval).
`phase_ps`	Phase offset in picoseconds. `0` if no phase offset is specified.
`jitter_pp_ps`	Peak-to-peak jitter in picoseconds. `0` if jitter is not enabled for this clock. See Simulation Specification Section 2.2.1.
`jitter_sigma_ps`	Jitter standard deviation in picoseconds (`jitter_pp_ps / 6`). `0.0` if jitter is disabled.
`drift_max_ppm`	Maximum drift in parts per million as configured via `--drift`. `0.0` if drift is not enabled. See Simulation Specification Section 2.2.2.
`drift_actual_ppm`	Actual drift value selected at simulation start from a Gaussian distribution clamped to `±drift_max_ppm`. Positive values mean the clock runs fast; negative means slow. `0.0` if drift is disabled.

Column	Description
`drifted_period_ps`	Drift-adjusted full period in picoseconds as a floating-point value (`period_ps × (1 + drift_actual_ppm / 1,000,000)`). Stored as `REAL` to preserve sub-picosecond precision from accumulated drift. `0.0` if drift is disabled.

Notes:

- The `clocks` table is written once at simulation start and is not updated during simulation.
- Clock names in this table correspond to signal names in the `signals` table where `type = 'clock'` and `scope = 'clocks'`.
- Viewers can use this table to display clock properties (frequency, jitter bounds, drift) without parsing metadata keys or computing values from signal transitions.
- For backward compatibility, readers should tolerate the absence of this table in older JZW files.

4. VALUE ENCODING

Signal values are stored as text strings in the `value` column of the `changes` table. The encoding depends on the signal width:

4.1 Single-Bit Signals

Single-bit signals (width 1) are stored as a single character:

Value	Meaning
`0`	Logic low
`1`	Logic high
`z`	High impedance (tri-state only)

4.2 Multi-Bit Signals

Multi-bit signals (width > 1) are stored as binary strings of `0`, `1`, and `z` characters, MSB first. The string length equals the signal width.

Examples:

Width	Value	Meaning
8	`10101010`	`8'hAA`
4	`0000`	`4'h0`
3	`1zz`	3 bits: MSB is 1, two LSBs are high-impedance

4.3 No X Values

Consistent with JZ-HDL semantics (Simulation Specification Section 2.5), `x` never appears as a runtime value. All storage is initialized to concrete `0`/`1` bits via seed-based randomization. The `x` character is not a valid value encoding in JZW files.

5. ANNOTATION DIRECTIVES

Annotation directives are source-level constructs that generate entries in the JZW `annotations` table during simulation. They have no effect on VCD or FST output.

5.1 @alert

Syntax:

```
@alert(<condition>, <color>)
@alert(<condition>, <color>, "<message>")
```

Creates an annotation of type `alert` when `<condition>` evaluates to true (non-zero) at the current simulation time.

- `<condition>` is a testbench `WIRE` identifier or hierarchical signal reference.
- `<color>` is a color name (Section 5.4).
- `<message>` is an optional string literal describing the alert.
- If `<condition>` is a multi-bit signal, it is truthy if any bit is 1.
- The alert fires after combinational settling, at the same evaluation point as `@print_if`.
- If the condition is true on consecutive ticks, an annotation is created for each tick where the condition holds.

Example:

```
@alert(full, RED, "FIFO full")
@alert(overflow, RED)
```

Behavior during `@run`: When an `@alert` directive is in scope, the simulator evaluates its condition at every tick during `@run`, `@run_until`, and `@run_while` execution. This is distinct from `@print`, which executes only at the point it appears in the simulation sequence.

5.2 @mark

Syntax:

```
@mark(<color>)
@mark(<color>, "<message>")
```

Creates a global annotation of type `mark` at the current simulation time. Marks are unconditional — they fire exactly once at the simulation time where they appear in the source.

- `<color>` is a color name (Section 5.4).
- `<message>` is an optional string literal.
- The `signal_id` in the annotations table is NULL (marks are global, not signal-specific).

Example:

```
@update {
    rst_n <= 1'b1;
}
@mark(GREEN, "Reset released")
```

5.3 @select

Syntax:

```
@select(<signal>, <color>)
```

Creates a range annotation of type `select` that highlights a signal for the duration of the next `@run` directive. The annotation's `time` is the current simulation time, and `end_time` is the simulation time after the following `@run` completes. Multiple consecutive `@select` directives are allowed before that run; each creates its own range annotation over the same upcoming run window.

- `<signal>` is a testbench `WIRE` identifier or hierarchical signal reference.
- `<color>` is a color name (Section 5.4).
- The `signal_id` is set to the referenced signal's ID.
- `@select` must be followed by either another `@select` or by a `@run`, `@run_until`, or `@run_while` directive. A compile error is reported if a `@select` chain is followed by any other directive.

Example:

```
@select(data_out, YELLOW)
@select(status, CYAN)
@run(ns=100)
// Highlights data_out and status for the same 100ns window
```

5.4 Trace Annotations

The `@trace` directive (Simulation Specification Section 4.9) generates annotations of type `trace` in the JZW database. These are automatically created by the simulator when `@trace(state=on)` or `@trace(state=off)` is encountered.

Column	Value
`type`	`"trace"`
`time`	Simulation time when trace state changed (picoseconds).
`signal_id`	NULL (trace is global, not signal-specific).
`message`	`"on"` or `"off"`.
`color`	NULL.
`end_time`	NULL.

Viewers should use trace annotations to visually indicate periods where waveform recording was disabled (e.g., grayed-out regions, gap markers, or collapsed time ranges).

5.5 Color Names

The following color names are valid for annotation directives:

Name	Intended Use
`RED`	Errors, violations, critical alerts

Name	Intended Use
`ORANGE`	Warnings, caution
`YELLOW`	Highlights, attention
`GREEN`	Success, milestones
`BLUE`	Informational markers
`PURPLE`	Debug, custom grouping
`CYAN`	Timing, clock-related
`WHITE`	Neutral

Color names are case-insensitive in source but stored as uppercase in the database. Viewers map these names to specific RGB values at their discretion.

5.6 Annotation Placement Rules

Rule	Description
ANN-001	`@alert` must appear at the top level of a `@simulation` block (not inside `@setup`, `@update`, or `@new`).
ANN-002	`@mark` must appear at the top level of a `@simulation` block (not inside `@setup`, `@update`, or `@new`).
ANN-003	`@select` must be followed by another `@select`, `@run`, `@run_until`, or `@run_while`; the chain must terminate in a run directive.
ANN-004	`@alert` condition must reference a signal in scope (declared in `WIRE`, `CLOCK`, or `TAP`).
ANN-005	`@select` signal must reference a signal in scope (declared in `WIRE`, `CLOCK`, or `TAP`).
ANN-006	Color name must be one of the predefined names (Section 5.4).

6. WRITE PERFORMANCE

6.1 Transaction Batching

The simulator must batch value-change and annotation inserts within transactions to achieve acceptable write performance. The recommended strategy is to wrap each tick's inserts (all value changes and annotations at a single time point) in one transaction. Alternatively, the simulator may batch across multiple ticks (e.g., every 1000 ticks) for higher throughput, at the cost of increased latency for real-time viewers.

6.2 Prepared Statements

The simulator should use prepared statements for the `INSERT INTO changes` and `INSERT INTO annotations` operations, as these are executed millions of times during a typical simulation.

6.3 Pragmas

The following pragmas are recommended for write performance:

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = -8000;        -- 8MB cache
PRAGMA temp_store = MEMORY;
```

7. CLI USAGE

7.1 Output Format Selection

```
jz-hdl --simulate sim_file.jz --jzw               # produces sim_file.jzw
jz-hdl --simulate sim_file.jz --jzw -o output.jzw  # explicit output path
```

Format flags are mutually exclusive; each simulation run produces exactly one waveform output format.

If no format flag is specified, the default output format is VCD (unchanged from the Simulation Specification).

8. QUERYING EXAMPLES

Because JZW files are standard SQLite databases, they can be queried directly from the command line or any SQLite client.

List all signals:

```
sqlite3 output.jzw "SELECT id, scope, name, width, type FROM signals;"
```

Get all value changes for a signal in a time range:

```
sqlite3 output.jzw "
  SELECT time, value FROM changes
  WHERE signal_id = 3 AND time BETWEEN 50000 AND 100000
  ORDER BY time;
"
```

Get all alerts:

```
sqlite3 output.jzw "
  SELECT a.time, s.name, a.message, a.color
  FROM annotations a
  LEFT JOIN signals s ON a.signal_id = s.id
  WHERE a.type = 'alert'
  ORDER BY a.time;
"
```

List all clocks with their properties:

```
sqlite3 output.jzw "
  SELECT name, period_ps, phase_ps, jitter_pp_ps, drift_actual_ppm
  FROM clocks;
"
```

Get the value of all signals at a specific time (most recent change at or before T=50000ps):

```
sqlite3 output.jzw "
  SELECT s.scope, s.name, c.value
  FROM signals s
  JOIN changes c ON c.signal_id = s.id
  WHERE c.time = (
    SELECT MAX(c2.time) FROM changes c2
    WHERE c2.signal_id = s.id AND c2.time <= 50000
  );
"
```

9. EXAMPLE

Given the simulation from Simulation Specification Section 7, using annotation directives:

```
@simulation fifo_ctrl
    @import "fifo.jz";

    CLOCK {
        clk_wr = { period=10.0 };
        clk_rd = { period=25.0 };
    }

    WIRE {
        rst_n [1];
        wr_en [1];
        rd_en [1];
        data_in [8];
        data_out [8];
        full [1];
        empty [1];
    }

    TAP {
        dut.wr_ptr;
        dut.rd_ptr;
    }

    @new dut async_fifo {
        clk_wr [1] = clk_wr;
        clk_rd [1] = clk_rd;
        rst_n  [1] = rst_n;
        wr_en  [1] = wr_en;
        rd_en  [1] = rd_en;
        din    [8] = data_in;
        dout   [8] = data_out;
        full   [1] = full;
        empty  [1] = empty;
    }

    // Alert whenever FIFO is full
    @alert(full, RED, "FIFO full")

    @setup {
        rst_n   <= 1'b0;
        wr_en   <= 1'b0;
        rd_en   <= 1'b0;
        data_in <= 8'h00;
    }
```

```
    @run(ns=50)

    @update {
        rst_n <= 1'b1;
    }
    @mark(GREEN, "Reset released")

    @run(ns=50)

    @update {
        wr_en   <= 1'b1;
        data_in <= 8'hAA;
    }
    @mark(BLUE, "Write burst start")

    @select(data_out, YELLOW)
    @run(ns=10)

    @update {
        data_in <= 8'hBB;
    }
    @run(ns=10)

    @update {
        wr_en <= 1'b0;
        rd_en <= 1'b1;
    }

    @run(ns=100)
@endsim
```

This simulation produces a `.jzw` file containing:

- **`meta` table:** 10 rows with simulation context (version, seed, tick resolution, etc.).
- **`signals` table:** 9 rows (2 clocks, 5 wires, 2 taps).
- **`clocks` table:** 2 rows (one per clock: `clk_wr` with period 10000 ps, `clk_rd` with period 25000 ps), plus jitter/drift columns if `--jitter` or `--drift` flags were used.
- **`changes` table:** 9 rows at time 0 (one per signal), plus one row for each subsequent value change.
- **`annotations` table:** Entries for the `@mark` directives at reset release and write burst start, the `@select` on `data_out`, and any ticks where the `@alert` condition on `full` was true.